Institutional Repository - Research Portal
Dépôt Institutionnel - Portail de la Recherche
researchportal.unamur.be
University of Namur

RESEARCH OUTPUTS / RÉSULTATS DE RECHERCHE

Hyperparameter Control Using Fuzzy Logic

Roy, Nicolas; Beauthier, Charlotte; Mayer, Alexandre

Published in:
Evolutionary computation

DOI:
10.1162/evco_a_00353

Publication date:
2025

Document Version
Version created as part of publication process; publisher's layout; not normally made publicly available

Link to publication

Citation for pulished version (HARVARD):
Roy, N, Beauthier, C & Mayer, A 2025, 'Hyperparameter Control Using Fuzzy Logic: Evolving Policies for Adaptive Fuzzy Particle Swarm Optimization Algorithm', *Evolutionary computation*, vol. 33, no. 2, pp. 279-308. https://doi.org/10.1162/evco_a_00353

General rights
Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal ?

Take down policy
If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.

Download date: 14. Oct. 2026

Hyperparameter Control Using Fuzzy Logic: Evolving Policies for Adaptive Fuzzy Particle Swarm Optimization Algorithm

Nicolas Roy nicolas.roy@unamur.be
Department of Physics, University of Namur, Namur, 5000, Belgium
Namur Institute for Complex Systems (naXys), University of Namur, Belgium

Charlotte Beauthier charlotte.beauthier@cenaero.be
Minamo Developpement, Cenaero, Gosselies, 6041, Belgium

Alexandre Mayer alexandre.mayer@unamur.be
Department of Physics, University of Namur, Namur, 5000, Belgium
Namur Institute for Complex Systems (naXys), University of Namur, Belgium

Abstract

Heuristic optimization methods such as Particle Swarm Optimization depend on their parameters to achieve optimal performance on a given class of problems. Some modifications of heuristic algorithms aim at adapting those parameters during the optimization process. We present a framework to design such adaptation strategies using continuous fuzzy feedback control. Fuzzy feedback provides a simple interface where probes are sampled in the optimization process and parameters are fed back to the optimizer. The probes are turned into parameters by a fuzzy process optimized beforehand to maximize performance on a training benchmark. Utilizing this framework, we systematically established 127 novel Fuzzy Particle Swarm Optimization algorithms featuring a maximum of 7 parameters under fuzzy control. These newly devised algorithms exhibit superior performance compared to the traditional PSO in the single-objective bound-constrained numerical optimization competition of CEC 2020. Additionally, two specific controls, highlighted for their efficacy and dependability, demonstrated commendable performance in real-world scenarios from CEC 2011.

Keywords

Particle Swarm Optimization, Systematic Algorithm Design, Fuzzy Control, Swarm Intelligence, Hyperheuristics

©2023 by the Massachusetts Institute of Technology

1 Introduction

Evolutionary Optimization (EO) algorithms aim at finding solutions of increasing quality to complex black-box problems. The quality of a solution is evaluated by the fitness function, an abstraction of the problem that takes a solution as input and outputs a figure of merit. Black-box fitness functions are EOs' primary use cases. A black-box has no analytical expression nor derivatives. Evolutionary optimizers sample (most often stochastically) the space of candidate solutions of the problem to find the pertinent extrema of the fitness function. These optimizers outperform pure random search by taking advantage of a heuristic. Classical examples are the Genetic Algorithm (GA), Covariance Matrix Adaptation Evolution Strategy (CMA-ES) and Particle Swarm Optimization (PSO) (Kennedy and Eberhart (1995)). However, optimal deployment of these evolutionary algorithms relies on meticulous configuration of a hyperparameter set to which they often demonstrate heightened sensitivity (van den Bergh and Engelbrecht (2006); Bartz-Beielstein et al. (2002)).

The study focuses on PSO whose optimal hyperparameter values vary over time as focus shifts from exploration to exploitation (Leonard and Engelbrecht (2013)). This is discussed in Shi and Eberhart (2001) where the PSO's inertia is compared to the temperature in simulated annealing. In this context, parameter tuning, parameters that we optimize once and for all, are insufficient. Therefore, parameter control methods (Eiben et al. (1999); Xin et al. (2009)) were developed to modify these parameters during an optimization process. The goal is twofold: reducing the number of sensitive parameters while increasing performance. However, Harrison et al. (2016) showed that numerous parameter control policies not only increase the number of free parameters: they are outperformed by constant or random parameter controls. In fact, Harrison et al. (2016) underscores the frequently increased number of parameters alongside the implementation of parameter control policies. This result can be understood for deterministic parameter controls given the emergent, non linear and sometimes chaotic nature of population-based heuristics. Time-varying parameter schedules like the ones reviewed in Bansal et al. (2011) for inertia, should suffer from the same limitations than parameter tuning (Eiben et al. (1999)).

Furthermore, weaknesses were also highlighted in adaptive control schemes (Harrison et al. (2018)) and a later study also showed that the empirical convergence criterion from Cleghorn and Engelbrecht (2014, 2015) is violated by most self-adaptive PSOs (Harrison et al. (2016)). This criterion was shown to be correlated to the convergence criterion from dynamic system theory (Poli (2009); Trelea (2003)). All these weaknesses illustrate the principles of the No Free Lunch (NFL) theorems (Wolpert and Macready (1997)) and provide a grim picture of parameter control for PSO. Yet, adaptive parameter control aims to better cope with the NFL challenge by readjusting continuously the heuristic to the problem at hand (Doerr and Doerr (2020); Eiben et al. (1999)). Adaptive methods differ from static (i.e. fixed) parameter control by reacting to various present or past aspects of the optimizer and its components, designated as *search situation* in Nickabadi et al. (2011). This feedback-driven approach effectively makes the heuristic strategy a function of the problem itself and has all the tools needed to assign optimal parameters to the swarm.

This article tackles the previously mentioned challenges by implementing a systematic benchmark-driven approach for the formulation of adaptation policies. The goal is to minimize the reliance on expert-tuned parameters, aiming to create a convergent PSO algorithm that is tunable for any target application. The proposed framework extends Fuzzy Particle Swarm Optimization (FPSO) from Shi and Eberhart (2001).

FPSO exhibits the issues highlighted by Harrison et al. (2016): the whole fuzzy feedback specification has been tuned by hand and its performance evaluated on only three functions (Tian and Li (2009); Shi and Eberhart (2001)). Nonetheless, the method remains appealing due to the complex, fine-grained control, the robustness, and expressiveness of fuzzy control.

The framework systematically optimizes the core components of the fuzzy feedback to maximize performance over a benchmark. This systematic design of fuzzy rulesets, provides the means to govern up to seven parameters. This task would prove challenging even for an expert, given the difficulty in identifying effective control schemes. While the framework has been implemented and tested exclusively for PSO, its applicability extends to any other population-based heuristic.

The article is organized as follows. Section 2 describes the operation and optimization of the fuzzy feedback. Section 3 assesses our most effective controllers using a comprehensive benchmark that includes various popular benchmark functions and the functions from the CEC 2020 competition on continuous, bound-constrained, optimization. Section 4 discusses and validates the behavior of the produced adaptive optimizers regarding their convergence and choice of parameters. Finally, in Section 5, two controllers face the CEC 2011 real world function set.

2 Systematic design of fuzzy feedback engines

Following the taxonomy from Doerr and Doerr (2020), our feedback falls between the state-dependent and learning-inspired parameter controls. These are subclasses of the wider adaptive control class from Eiben et al. (1999). The parameter values are dynamically adjusted during the runtime, contingent upon the past and current state of the optimizer. The feedback engine integrates mainly like the problem does in an optimization process, as depicted in Figure 1(A). The inputs of the feedback are scalar probes established by the implementation and its outputs are adapted parameters for the next iteration. The details of the inference process are presented in Section 2.2.

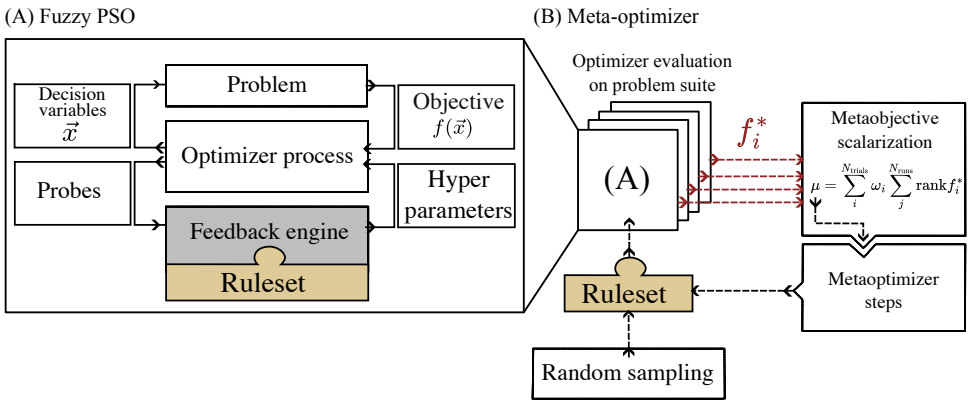


Figure 1: **Fuzzy feedback engine integration and meta-optimization.**

The optimization process illustrated in Figure 1(B), labeled as a meta-optimizer, operates within a rule space denoted as $\mathcal{R}$. Each rule within this space corresponds to a unique optimizer, emphasizing the meta-level optimization of feedback rules. The figure of merit minimized by the meta-optimizer is the performance of the optimizer on

a target problem suite while utilizing the feedback. The meta-optimization is steered by a categorical optimizer operating on the space $\mathcal{R}$. The first ruleset is sampled randomly in $\mathcal{R}$ while following rulesets are sampled using either Bayesian optimization (Močkus (1975)), GA or even PSO from Wang et al. (2021). In this study, we used the PSO algorithm itself. The categorical variables are managed by the implementation of Wang et al. (2021).

A meta-objective expresses the performance of the optimizer on a training benchmark suite. For each evaluation of an FPSO ruleset's meta-objective, the whole training suite is evaluated five times with various random seeds. The objective values f_{jk}^{best} obtained for function j at run k are then converted to normalized ranks using an archive of all previous evaluations of this benchmark. This conversion to ranks helps with handling the broad range of fitness values amongst the benchmark. Once averaged on the N_{runs} executions, they provide an average rank $\bar{r}_j$ for each of the N_{trials} functions of the training suite. These multiple objectives are summed up to a single scalar objective μ with an optional importance factor ω_j:

$$\mu = \sum_{j}^{N_{\text{trials}}} \omega_j \bar{r}_j, \text{ where } \bar{r}_j = \sum_{k}^{N_{\text{runs}}} \text{rank} f_{jk}^{\text{best}}.$$

In our experience, this simple scalarization combined to ranks proved to be more reliable than a true multi-objective approach.

2.1 Particle Swarm Optimization

In this work, we apply the framework to PSO, given its popularity, robustness and simplicity. This section will introduce critical PSO parameters that will be exposed to the control of the fuzzy feedback. The PSO algorithm provides a well-known basis (Zambrano-Bigiarini et al. (2013)) on which to showcase our feedback meta-optimizer.

Craig Reynolds' Boids inspired the original PSO authors, Kennedy and Eberhart (1995), to create an optimizer leveraging swarm intelligence. The link between those algorithms is the exploitation of weak emergence (Bedau (1997)) in order to achieve complex behavior at the swarm's level. Weak emergence, in particular, describes emergence as a bottom-up process: simple interacting agents following simple rules produce emergent complexity that is observable at a larger scale.

The swarm in PSO is a collection of n_S particles flying in a search space of dimension n_D. Each particle holds a position, representing the n_D decision variables, along with its corresponding objective or fitness score. Each particle also memorizes its best-known position and fitness score.

Each particle i follows the same behavior. The position of a particle is incremented at each step t by its speed:

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t). \tag{1}$$

The heuristic resides in the definition of the speed:

$$\mathbf{v}_i(t+1) = c_1\mathbf{r}_{1i}(t) \circ (\mathbf{m}_i(t) - \mathbf{x}_i(t)) + c_2\mathbf{r}_{2i}(t) \circ (\mathbf{g}(t) - \mathbf{x}_i(t)) + \omega\mathbf{v}_i(t), \tag{2}$$

with the operator $\circ$ representing a Hadamard (element-wise) product. The particle is perpetually guided towards two weighted attractors: the memory $\mathbf{m}_i$ of its best-known previous position and the best position, $\mathbf{g}$, ever occupied by a particle in the entire swarm. The weights are $c_1\mathbf{r}_{1i}$ for the memory of the best position encountered by particle i and $c_2\mathbf{r}_{2i}$ for the global best. The vectors $\mathbf{r}_{1i}$ and $\mathbf{r}_{2i}$ are sequences of real

random numbers sampled uniformly in $[0, 1]$. The third term of the equation implements inertia, one of the earliest and most widespread improvements to PSO from Shi and Eberhart (1998). A particle's velocity is based on a fraction ω of its previous value. This extra parameter ω facilitates tuning of exploration.

Later in this work, we will also refer to a modification of PSO by Juan et al. (2004): Quantum Particle Swarm Optimization (QPSO). This modification of PSO updates the position of the particle following

$$\mathbf{x}_i(t+1) = \mathbf{a}_i(t) \pm \frac{\|\mathbf{x}_i(t) - \mathbf{a}_i(t)\|}{g} \circ \ln \frac{1}{\mathbf{u}}, \tag{3}$$

with $\mathbf{u}$ a sequence of random variables sampled uniformly in $]0, 1]$ and $g = 0.96$ as in Juan et al. (2004). The particle is sampled around the position $\mathbf{a}_i$ with a probability density following

$$Q(\mathbf{x}) = \frac{1}{2} \frac{g}{||\mathbf{x}_i(t) - \mathbf{a}_i(t)||} e^{-g \frac{||\mathbf{x}||}{||\mathbf{x}_i(t) - \mathbf{a}_i(t)||}}.$$

The variant is called quantum because this probability density echoes the quantum delta well system in physics. The mean position $\mathbf{a}_i$ for the sampling is given by

$$\mathbf{a}_i(t) = (\mathbf{r}_1 \circ \mathbf{m}_i(t) + \mathbf{r}_2 \circ \mathbf{g}(t)) \oslash (\mathbf{r}_1 + \mathbf{r}_2), \tag{4}$$

with $\oslash$ the Hadamard (element-wise) division.

Another commonly used addition to PSO is the reduction of the global best $\mathbf{g}$ to a local best $\mathbf{l}_i$ by implementation of a social topology. Topology aims at improving the exploration capabilities of the swarm by limiting the exposure to local minima. The topology thus prevents these minima from attracting the whole swarm in premature convergence. A topology is an oriented graph representing the information-sharing paths in the swarm, much like a social network, where only first neighbors can share their experiences. The local best agent is referred to as the best friend, which is chosen for its better memory amongst the neighborhood of each agent in the graph. For example, the original algorithm's topology is a fully connected graph; i.e., each agent is a neighbor of all other agents. In our implementation, we worked with a random adaptive topology like described in Zambrano-Bigiarini et al. (2013) following an Erdős-Rényi graph from Erdős and Rényi (1959). Random adaptive means that the topology is reconstructed when swarm stagnation occurs. This straightforward and flexible topology was particularly fit for our framework as we aim to control K, the graph's connectivity.

2.2 Fuzzy Logic

Fuzzy logic lies at the core of our framework, allowing to link a representation of the state of the optimizer to new parameters for the next iteration. In this section, we provide the reader with an overview of key concepts relevant to our implementation like *linguistic variables*, *rules* and *(de)fuzzification*. The actual implementation will find a dedicated section later.

Fuzzy Logic is a widely used tool in process control given its robustness against uncertainty and fluctuations of the antecedent. It is an alternative to Aristotelian logic that relies on linguistic control rules. Boolean truth values are replaced by fuzzy truth levels, often expressed as real numbers in $[0, 1]$. An example of the interpretations of a scalar value T by classic and fuzzy logic is given in Figure 2.

The first step when designing fuzzy control is to define all variables used in the Fuzzy Inference Engine (FIE). Each variable is represented in the FIE by what is called

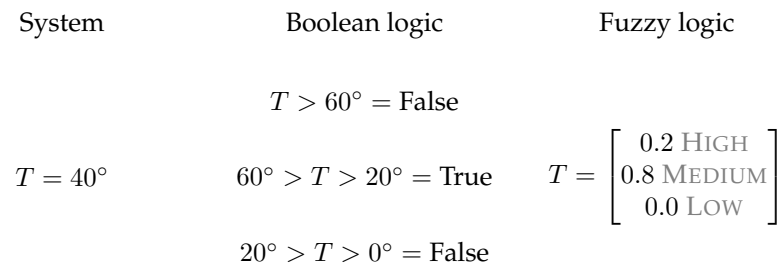


Figure 2: Examples of boolean and fuzzy logic interpretation of a variable T expressing temperature as LOW, MEDIUM or HIGH.

a linguistic variable. The variable also possesses a scalar value in the system to be controlled. A linguistic variable is a variable whose values are words or sentences in a natural or synthetic language, cited from Zadeh (1988). As an example, linguistic values for temperature could be LOW, MEDIUM, and HIGH.

In our case, we choose to work with a common basis of linguistic values for all three variables defined as:

$$\mathcal{L} = \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}\,.$$

Scalar representation of the variable is turned into a fuzzy state by *fuzzyfication*. During this process the scalar value is converted to a vector of truth levels, one for each value in $\mathcal{L}$. For each value $p_i \in \mathcal{L}$ of a linguistic variable with n values, the translation requires the definition of a truth function. With input value r_A in $\mathbb{R}$, the functions for an arbitrary linguistic variable A are

$$\mathcal{T}_i : \mathbb{R} \to [0, 1] \tag{5}$$

$$r_\text{A} \mapsto \mathcal{T}_i(r_\text{A}; \mathbf{a}), \tag{6}$$

with $\mathbf{a}$ a vector of parameters for the functions $\mathcal{T}_i,\ i = 1, \ldots n$, that defines linguistic variable A.

The truth function gives the tendency of the variable to be LOW, MEDIUM or HIGH. Trapezoid, sigmoid or Gaussian profiles are typical choices for such functions. We chose to work with ramps and triangles for simplicity as shown in Figure 3. In our implementation, the same set of functions $\mathcal{T}_i$ is used for all variables: two ramps for LOW and HIGH and a triangle for MEDIUM. The distinction only comes from the vector of parameters associated to each variable. The vector of three parameters defines key values for these functions, as shown in Figure 3.

In summary, a linguistic variable A with n values will be encoded by a fuzzification function F formed by n truth functions characterized by an array of parameters $\mathbf{a}$:

$$F : \mathbb{R} \to [0, 1]^n \subset \mathbb{R} \tag{7}$$

$$r_\text{A} \mapsto F(r_\text{A}; \mathbf{a}) \triangleq \rho^A = \begin{bmatrix} \mathcal{T}_1(r_\text{A}, \mathbf{a}) \\ \vdots \\ \mathcal{T}_n(r_\text{A}, \mathbf{a}) \end{bmatrix}. \tag{8}$$

Now that the process of encoding fuzzy variables is presented, we can explain the inference process. Rule sets are expressed as follows; using three example variables:

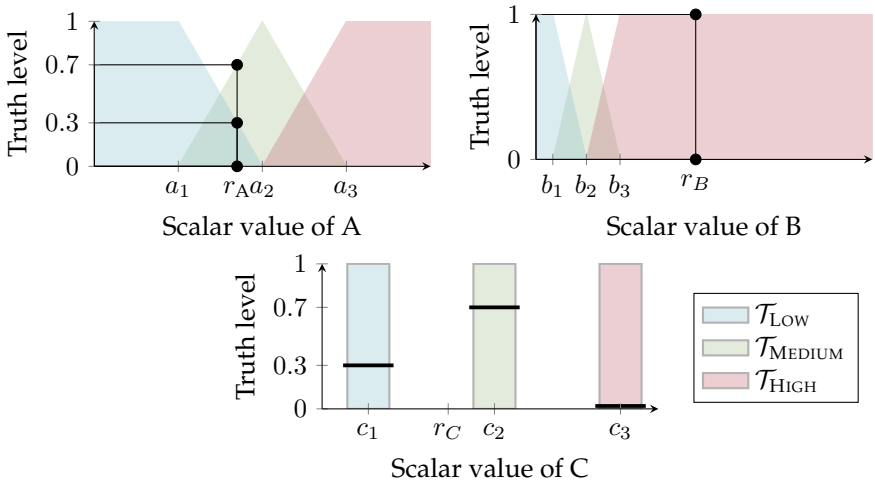


Figure 3: **Sample truth functions.** The three linguistic values LOW, MEDIUM and HIGH are represented in blue, green and red. The fuzzification process is illustrated on input variable A while the Sugeno-Kang interpretation of an output variable during defuzzification is shown on the bottom graph (C).

if A is HIGH or B is LOW then C is HIGH

if A is MEDIUM or B is MEDIUM then C is MEDIUM

if A is LOW and B is HIGH then C is LOW

Statements like A is LOW refer to the element ρ^A_{LOW} which is a scalar truth level equal to 0.3 in Figure 3(A). Operators used in our case are represented in Table 1. Each logical operator is substituted by its fuzzy, real counterpart. Priority of operations depends on the placement of parentheses which is part of the definition of our rule sets. Each of the three lines shown above corresponds to an output truth level for an output variable C, forming the associated fuzzy state ρ^C. This fuzzy state is illustrated in Figure 3(C) by the thick black lines.

Boolean	Fuzzy (Zadeh)
u **and** v	$\min\{u, v\}$
u **or** v	$\max\{u, v\}$

Table 1: **Mapping between Boolean and Zadeh operators from Zadeh (1965).**

Once a variable such as C is expressed in the linguistic basis, fuzzy logic also provides us with a way to produce a parameter value using a defuzzification function D

$$D : [0, 1]^n \to \mathbb{R} \tag{9}$$

$$\rho^C \mapsto D(\rho^C; \mathbf{c}) = r_{\text{C}}. \tag{10}$$

Here, $\mathbf{c}$ is another set of parameters. This step is required as the output value is the parameter to be fed back to the optimizer.

A commonly used inference method is Mamdani Inference, discussed in Izquierdo and Izquierdo (2018). In this case, D is defined through centroids computation, and is quite expensive numerically. In our implementation we followed instead Takagi-Sugeno-Kang inference from Sugeno (1974). This method is also called Sugeno integral: a weighted average of the parameters $\mathbf{c}$ (that define the membership functions), with the weights found in the membership vector ρ^C:

$$r_{\mathrm{C}} = \frac{\sum_{i=0}^{n} \rho_i^C c_i}{\sum_{i=0}^{n} \rho_i^C}. \tag{11}$$

In this expression we require a single parameter c_i per membership function, the *key value* of the associated linguistic value. Therefore, the specific shape of membership functions does not matter as shown in Figure 3(C) and equation 11.

We now have a scalar variable r_{C} that can be fed back in the controlled system. This section allowed us to review the full process of turning input scalar values into fuzzy states. The state is linked to a linguistic variable, defined by simple membership functions. A state for an output variable C was produced using Zadeh operators, the state from which we finally extracted the scalar value using Sugeno inference.

By using some probes from the controlled system as inputs and a parameter as output, we are able to provide the needed feedback for the parameter. As many parameters as needed can be controlled in this way, using the same probes with different rules. The whole process defines our Fuzzy Inference Engine.

2.3 Implementation of fuzzy control for PSO

This section introduces the variables involved in our fuzzy controller. These are split in two groups: input probes and output parameters. The probes are empirical indicators of the optimizer's state and parameters are the ones of the heuristic algorithm (here PSO). As detailed in the preceding section, the encoding of these variables requires the use of membership functions. Adhering to the principle of parsimony (Hawkins (2004)), our selection of functions is geared towards achieving the utmost simplicity: two half trapezoids on the sides of the domain and a triangle in the center, as illustrated in Figure 4. The key values defining those functions are referred to as a_{LOW}, a_{MEDIUM} and a_{HIGH} in Figure 4 and Table 2.

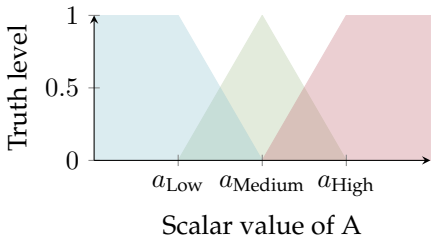


Figure 4: **Membership functions used in this study.**

2.3.1 Probes

The probes should be scarce and give meaningful information about the optimizer's status. Therefore, we avoid both redundant and meaningless probes. Redundant

Symbol	Name	a_{LOW}	a_{MEDIUM}	a_{HIGH}
ϕ	Relative improvement	-1.0	0.0	1.0
δ	Distance to best friend	0.0	0.03	0.06
t	Iterations	0.0	0.3	0.7

Table 2: **Parameters for the definition of each fuzzy probe.** The three parameters of the *parameters vector*, cf. Section 2.2, that define our truth-level functions (see illustration in Figure 4).

probes would be the manifestation of similar metrics in the optimization process. Without being harmful, they still bring complexity without usefulness. Meaningless probes are those that are too noisy, diverge to infinity or converge quickly to a constant value; i.e. zero whatever the optimization process.

The first probe, called *Relative Improvement* or ϕ, inquires about the performance of the agent i. It is closely related to the fitness value, as seen in its definition below:

$$\phi_i(t) = -\frac{4}{n_S}\ \textbf{clamp}\Big[\text{rank}(f_i(t)) - \text{rank}(f_i(t-1)) + \ c, -\frac{n_S}{4}, \frac{n_S}{4}\Big],$$

$$\textbf{clamp}\,(x, \min, \max) = \begin{cases} \min & \text{if } x < \min \\ x & \text{if } \min \leq x \leq \max \\ \max & \text{if } x > \max \end{cases}$$

where n_S is the size of the swarm and $f_i(t)$ the fitness at iteration t. To address the concerns discussed earlier, ranking and clamping were implemented instead of relying on raw fitness variations. A cooperative component c is also present in the metric. The bonus $c = \pm\frac{n_S}{40}$ is positive or negative when the swarm's best-known point improves or stagnates. The bonus is doubled for the particle ($\mathbf{g}$ in Section 2.1) that enabled the swarm's improvement.

The second probe, called *Distance to Best Friend*, is tied to the position of the agent ($\mathbf{x}_i$) in the search space. It is defined as the Euclidean distance to the agent's best friend ($\mathbf{l}_i$) divided by the length of the diagonal of the search space.

$$\delta_i(t) = \frac{||\mathbf{l}_i(t) - \mathbf{x}_i(t)||}{||\mathbf{b}_u - \mathbf{b}_l||},$$

with $\mathbf{b}_u$ and $\mathbf{b}_l$ the upper and lower boundaries of the search space.

The last probe used in our implementation, called *Iterations*, is the fraction of the problem's evaluation budget spent. Unlike former probes, this one is common to all agents of the swarm. With a constant population size like in PSO, this probe is also the fraction of allowed iterations spent.

All key values for probes are listed in Table 2. Key values for ϕ and δ were defined empirically by observing their mean values and upper confidence bounds in classic PSO for various functions and number of dimensions. The exact value of these keys does not matter much as long as it remains representative of empirically observed values.

2.3.2 Parameters

Regarding the parameters to control, we used six standard parameters of PSO which are summarized in Table 3. Among those parameters, u and l are upper and lower limits placed on the euclidian norm of the speed of each particle. Their respective a_{LOW},

Symbol	Name	Lower Bound	Upper Bound
ω	Inertia	0.0	1.0
c_1	Cognitive coefficient	0.0	2.5
c_2	Social coefficient	0.0	2.5
h	Hybridization with Quantum PSO	0.0	1.0
l	Lower speed boundary	0.0	0.1
u	Upper speed boundary	0.0	1.0
K	Connectivity of topology	0.0	1.0

Table 3: **Optimization bounds for the key values of each fuzzy parameter.** The three key values are from the *parameters vector* discussed in Section 2.2. For the connectivity, the values are normalized by the size of the swarm (n_S). For the upper and lower speed limits (u, l), the values are normalized by the boundaries of each dimensions of the search space.

a_{MEDIUM} and a_{HIGH} key values were optimized as part of the meta-optimizer decision variables. These key values are encoded in a search space $\mathcal{M}$ as $[m_1, m_2, m_3] \in \mathcal{M}$ with $a_{\text{LOW}} = m_1 - m_2, a_{\text{MEDIUM}} = m_1, a_{\text{HIGH}} = m_1 + m_3$. The center of the range (m_1) respects boundaries of Table 3. Deviations m_2 and m_3 follow the same boundaries but are clamped to avoid $m_1 - m_2$ and $m_1 + m_3$ going out of these bounds.

In order to showcase a tentative use case of the framework in hyperheuristics (Burke et al. (2003); Cowling et al. (2002)), we also included a new parameter h. At each iteration, we override the classical PSO update equation in favor of QPSO from Juan et al. (2004) with a probability h.

2.4 Fuzzy ruleset dialect

As all the variables possess 3 linguistic values, we work with triplets of rules, one for each linguistic value of the output variable (LOW, MEDIUM, HIGH). We will only need to describe one rule, the triplets being formed by concatenation. Let us take for example a dummy LOW rule for parameter ω:

$$\omega \text{ is LOW: if } (\phi \text{ is HIGH and } \delta \text{ is MEDIUM}) \text{ or } t \text{ is IGNORE.}$$

This linguistic representation, while easy to read, has to be encoded like illustrated in Figure 5 for faster processing. We note the addition of an IGNORE value which allows the engine to ignore a probe, t in this case. The set of linguistic values common to all variables is identified to a set of integers using label encoding:

$$\mathcal{Q} = \{0 : \text{LOW}, 1 : \text{MEDIUM}, 2 : \text{HIGH}, 3 : \text{IGNORE}\}.$$

The label of each linguistic value present in the rule is encoded in binary form and placed in the green slot matching the different probes in Figure 5. The gray spaces are used to store the operators in a similar fashion, from the set:

$$\mathcal{O} = \{0 : \text{Or}, 1 : \text{And}\}.$$

With this representation, we ensure high simplicity and numerical performance while preserving enough generality. As mentioned before, we strive for simple controllers that are easier to optimize and make for relatively sensible controllers. A notable alternative to our method is the syntax tree which offers even more flexibility but is too complex and slower to process.

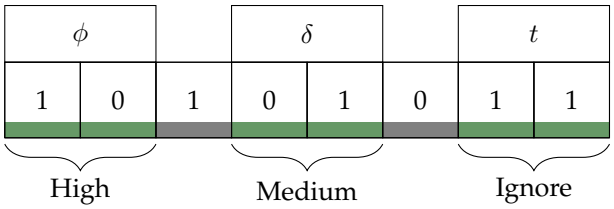


Figure 5: **Binary encoding of a single rule.** Linguistic values are stored in the green tiles and operators in the gray ones. Each green tile is associated to a different probe.

A rule is formed in this way for each linguistic value of the output parameter: the resulting triplet is placed in a 32-bits integer like in Figure 6. The remaining free byte is used to store a flag and the priority of operations for each rule (i.e. positioning of parentheses). Various associations are possible for the order of operations in a rule like Figure 5. Each possibility is again mapped to an integer:

$$\mathcal{P} = \begin{Bmatrix} 0 : (B \odot C) \odot A, \\ 1 : (C \odot A) \odot B, \\ 2 : (A \odot B) \odot C \end{Bmatrix} \tag{12}$$

with $\odot$ representing any of the two operators. Number 2 is used in the case of the rule from Figure 5.

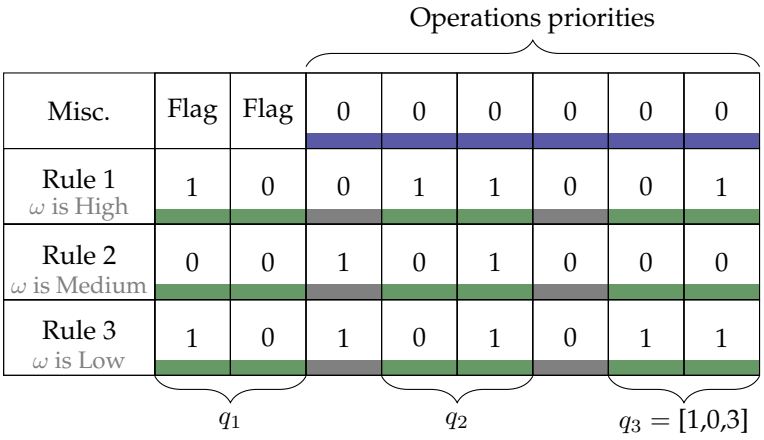


Figure 6: **Definition of a triplet of rules for ω.** The triplet of rules is stored in a 32-bits integer with the first byte dedicated to miscellaneous information while the three other bytes represent the three rules encoded like in Figure 5.

2.5 Formulating the rulesets as a set of combinations

The meta search space $\mathcal{R}$ is structured in order for the framework to be compatible with any meta-optimizer equipped to handle categorical and continuous variables. The search space is built on an alternative representation of the binary dialect. While the dialect can express all kinds of rules, it is not fit to treat the rulesets like decision variables in a search space. This representation leads to a single 32-bits integer variable per

rulseset on which it is not easy to impose constraints. Such constraints being necessary to prevent contradictions between encodable rules and tune the size of the meta search space.

Let $\bar{\mathcal{Q}}$ be the space of combinations of 3 out the 4 linguistic values in $\mathcal{Q}$. The rulesets are specified using three of its elements $q_i \in \bar{\mathcal{Q}}, i = 1 \dots 3$. These three combinations will each fill one green column in Figure 6 from top to bottom. The same logic applies for the set of operators $\mathcal{O}$ and its permutation set $\bar{\mathcal{O}}$. One method for adjusting the size of the search space involves regulating the occurrences of each linguistic value within these combinations. Specifically the number of times a value can be reused in a combination. For illustration, if we do not want to see HIGH appearing in two lines of the ruleset, we only need to set the number of reuse to 0 for this linguistic value.

The resulting meta search space, denoted as $\mathcal{R}$, encompasses nine dimensions for each controlled parameter. These dimensions include five for the rules, one for the priorities of operations and three for the membership functions. We express it by the Cartesian product

$$\mathcal{R} = \bar{\mathcal{Q}} \times \bar{\mathcal{O}} \times \bar{\mathcal{Q}} \times \bar{\mathcal{O}} \times \bar{\mathcal{Q}} \times \bar{\mathcal{P}} \times \mathcal{M}.$$

Here, $\bar{\mathcal{Q}}$ represents the set of combinations of 3 out of 4 linguistic values, $\bar{\mathcal{O}}$ denotes the set of combinations of 3 operators out of 2 (or, and) and $\bar{\mathcal{P}}$ contains the combinations of operations priorities. $\mathcal{M} \subset \mathbb{R}^3$ represents the key values of the parameter.

3 Tuning of a new Adaptive Fuzzy Particle Swarm Optimization algorithm

3.1 Benchmark problems

In order to optimize FPSO feedbacks using our framework, a specific set of problems needs to be targeted. Training and validation benchmarks are required in a way similar to the training of machine learning algorithms. In the following analysis, the framework optimizes fuzzy feedbacks on the training set before they are applied to several validation benchmarks.

The benchmarks considered are listed in Table 4. Three sets of functions are considered with various dimensions. Their choice is coherent with our objective: to provide evidence that the produced FPSO specialized on a training suite of problems has a wide enough performance window to be transferable to distinct but slightly related problems.

- Our own set of functions from Table 5 was designed for training: its covers a wide range of fitness landscapes with both unimodal and multimodal functions. It is partly inspired by Harrison et al. (2016). The function set was exploited with various numbers of dimensions listed in Table 4 as $\text{HAR}^{10,20,40,100}$.

- The function set from CEC 2020 Single Objective Bound Constrained Optimization Competition (Yue et al. (2020)) is considered for validation. This function set shares no function with our benchmark but some of its functions are a composition of these. Like above, the set has multiple dimensions listed in Table 4 as $\text{CEC}^{10,20,50}$.

- Real-world test-cases 1-7 and 9 from CEC 2011 (Das and Suganthan (2011)) are finally tackled. They provide new fitness landscapes and an opportunity to discuss practical use-cases of our framework. These are labeled RW and their dimensions are fixed for each problem definition.

ID	Function Set	Dimensions (n)	Max Evaluations
HAR^{10}	From Table 5	10	4e+4
HAR^{20}		20	
HAR^{40}		40	
HAR^{100}		100	
CEC^{10}	CEC_{2020}	10	
CEC^{20}		20	
CEC^{50}		50	
RW	CEC_{2011}	From 1 to 38	From 5e+4 to 15e+4

Table 4: **Single objective benchmarks composed of our function set, the CEC 2020 function set or the CEC 2011 set of real-world problems.** The benchmark used for training the optimizers is highlighted in grey.

3.2 Performance metrics

The performance metrics are borrowed from Harrison et al. (2016) where a comprehensive evaluation of PSO inertia (ω) controllers is provided. The two metrics are the accuracy and consistency. The accuracy is the fitness of the global-best particle at the conclusion of the optimization process. It is defined for each function j and independent run k of a benchmark as

$$\text{ACC}_{jk} = f_{jk}^{best}.$$

Consistency is quantified as the squared difference between the accuracy of each independent run (f_{jk}^{best}) and the average accuracy across all runs ($\bar{f}_{j}^{\text{best}}$):

$$\text{CONS}_{jk} = \left(f_{jk}^{best} - \bar{f}_{j}^{best}\right)^2.$$

ID	Name	Lower Bound	Upper Bound	Modal.	Reference
f_1	Absolute	-100.0	100.0	U	Harrison et al. (2016)
f_2	Ackley	-32.775	32.775	M	Adorio and January (2005)
f_3	Alpine 1	-10.0	10.0	M	Jamil and Yang (2013)
f_4	Egg holder	-512.0	512.0	M	Jamil and Yang (2013)
f_5	Elliptic	-100.0	100.0	U	Jamil and Yang (2013)
f_6	Griewank	-600.0	600.0	M	Adorio and January (2005)
f_7	Hyper ellipsoid	-5.12	5.12	U	Yue et al. (2020)
f_8	Michalewicz	0.0	3.14	M	Yue et al. (2020)
f_9	Norwegian	-1.1	1.1	M	Yue et al. (2020)
f_{10}	Quadric	-100.0	100.0	U	Yue et al. (2020)
f_{11}	Quartic	-1.28	1.28	U	Storn and Price (1995)
f_{12}	Rastrigin	-5.12	5.12	M	Adorio and January (2005)
f_{13}	Rosenbrock	-2.048	2.048	M	Rosenbrock (1960)
f_{14}	Saloman	-30.0	30.0	M	Jamil and Yang (2013)
f_{15}	Schaffer 6	-100.0	100.0	M	Jamil and Yang (2013)
f_{16}	Shubert	-10.0	10.0	M	Yue et al. (2020)
f_{17}	Sphere	-5.0	5.0	U	Adorio and January (2005)
f_{18}	Step	-100.0	100.0	M	Yue et al. (2020)
f_{19}	Vincent	0.25	10.0	M	Al-Roomi (2015)

Table 5: **Composition of the training functions set for $\text{HAR}^{10,20,40,100}$.** All functions are evaluated with variable number of dimensions n_D. Upper and lower bounds are indicated as well as the unimodal (U) or multimodal (M) quality of each function.

The accuracy and consistency obtained at the end of each independent run are then post-processed to provide a statistically significant measure of performance. Keeping consistency with Harrison et al. (2016), all methods undergo a structured tournament evaluation, wherein they are systematically compared in pairwise fashion. Each victory accrues one point, while each defeat incurs a loss of one point. For each function within the benchmark, the outcome of victory or defeat is determined through symmetric pairwise Wilcoxon-Mann-Whitney U tests (Wilcoxon (1945)) conducted at a significance level of 0.05. Upon reaching the significance level, the method exhibiting the superior median metric is declared the winner. The number of points obtained by a method is called score. All methods obtain a score in this way for both accuracy and consistency metrics and for each function. Finally, these scores are presented in three ways:

- The sums of scores Σ_{ACC} and Σ_{CONS} over all functions. This metric is sometimes split between unimodal and multimodal test cases with the indication Σ^U or Σ^M.
- The best rank frequencies BRF_{ACC} and BRF_{CONS} indicate on how many functions the method has the biggest score for the associated metric.
- The top-five rank frequencies: 5RF_{ACC} and 5RF_{CONS} are similar to BRFs but count occurrences of the method in the top-five instead of only accounting for the best.

3.3 Exploring the meta-optimization of feedback controller

The choice of parameters to be controlled is anticipated to influence the performance of the resulting FPSO. A selection of controlled parameters is referred to as control prototype and the number of controlled parameters as the size of the prototype. There are 7 parameters available in total. Each meta-optimization of a prototype minimizes the meta-objective μ described in Section 2. Different control prototypes lead to different values of the meta-objective.

In order to choose how many and which parameters will be controlled, all 127 possible prototypes are optimized and evaluated on benchmark HAR^{40}. In order to facilitate generalization from these results, meta-optimizations were reproduced in 20 independent runs and all the values presented are averaged on those 20 runs. Figure 7 summarizes the performance impact of both the presence of a parameter in a prototype and the size of the prototype on benchmarks HAR^{40} and CEC^{50}.

Figure 7(B) displays the frequency at which top-five accuracy is reached on HAR^{40} when controlling *at least* the parameters specified on the abscissa and ordinates. The diagonal elements cover prototypes containing *at least* the single parameter found in both abscissa and ordinates. For example, we can say from Figure 7(B) that 92% of the top-five prototypes for each function of HAR^{40} contain K. In fact, improved accuracy is realized through prototypes that include the inertia (ω) and/or the topology connectivity (K), and to a lesser degree, the upper speed limit (u). Figure 7(A) displays a positive evolution of accuracy and consistency scores for a prototype size going from zero (constant PSO) up to six. In fact, controlling five to six parameters leads to maximal accuracy and consistency but controlling all seven of them is less efficient. As we consider the training set, this indicates a limitation of either the meta-optimizer or an intrinsic mix-up between the seven varying parameters. In the following discussion, we will refer to prototypes $\text{FPSO}_1(\omega, c_1, u, K)$ and $\text{FPSO}_2(\omega, l, u, K)$, the two best prototypes on HAR^{40}. Both contain the most efficient parameters combinations we identified. $\text{FPSO}_3(\omega)$ will also appear since it stands as the second best single-parameter prototype on HAR^{40} (behind $\text{FPSO}(K)$) and is a thoroughly studied PSO parameter.

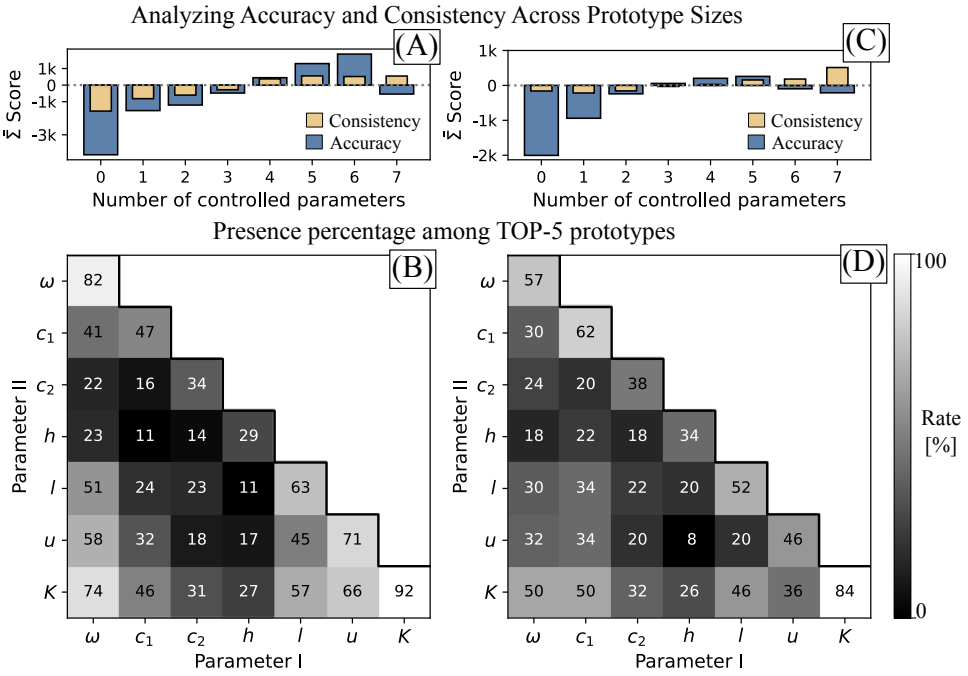


Figure 7: **Prototypes selection: the influence of parameters on accuracy and consistency on benchmarks HAR^{40} and HAR^{50}.** (A, C) Average accuracy on benchmarks HAR^{40} and CEC^{50} respectively for a varying number of controlled parameters. (B, D) The dependency between the inclusion of parameters in a prototype and the presence percentage among the top-five prototypes of each function.

Figures 7(C) and 7(D) show the exact same metrics for benchmark CEC^{50}. On this benchmark, the connectivity is still a clear leader among the parameters, being present in 84% of the benchmark's top-five. This dominance of parameter K is partly explained by the control ranges for connectivity being distributed around a mean of $0.42n_S$ (median of $0.48n_S$) with a standard deviation of $0.2n_S$. Both benchmarks tend to benefit from a higher adaptive topology connectivity than the one suggested by Zambrano-Bigiarini et al. (2013): $K = 3$ for a population size $n_S = 40$, or $K = 0.075n_S$. The average dynamic range of the controller is also large: $0.35n_S$. This highlights the benefits of more contrasted connectivity levels in a single optimization. Inertia stands-out a little less than on the training benchmark but is still well represented, particularly in conjunction with K. Parameter c_1 now has a performance similar to ω, showing a better transfer of optimization ability. The most interesting graph is Figure 7(C) where we see accuracy failing (more losses than victories) with more than five controlled parameters. On a validation benchmark, this behavior indicates that, when there are more than five parameters, the large controllers overfit the training problem suite (HAR^{40}). This poor performance is certainly expected from Figure 7(A) as the largest prototype also started to loose accuracy. The best prototypes on this CEC^{50} benchmark are FPSO$_4(\omega, c_1, l, K)$ and FPSO$_5(\omega, c_1, h, K)$. As expected from the analysis of Figure 7(D), c_1 appears in both prototypes while u disappears. We also remember FPSO$_6(K)$, the best single-parameter on this same benchmark.

Algorithm	Github Page (commit hash)	Parameters
PSO	kaeryv/Hybris$_{\text{7fbfb08}}$	$n_S = 40, \omega = 0.7298,$ $c_1 = 1.49618, c_2 = 1.49618$
PSO-RIW	kaeryv/FuzzySwarmBenchmarks	$\omega \in [0.5, 1.0]$
APSO-VI		$\omega_{\text{min}} = 0.3, \omega_{\text{max}} = 0.9, \delta\omega = 0.1$
GA	rmsolgi/geneticalgorithm$_{\text{e33726e}}$	$p_{\text{mutation}} = 0.1, p_{\text{crossover}} = 0.5,$ $n_S = 100, r_{\text{parents}} = 0.3$
L-SHADE	kaeryv/shady$_{\text{4f1daf6}}$	$n_S = 200$
CMA-ES	CMA-ES/pycma$_{\text{705bbd8}}$	$n_S = 15$ $\sigma = 0.6$

Table 6: **Reference algorithms.** The main parameters are listed as well as the Github urls and commits (revisions) used in our benchmarks. Other parameters were left to default values (see codes for more details). The population size for CMA-ES is chosen automatically by the code and took the value of 15 for $n_D = 40$ and 50 dimensions.

On a technical note, each meta-optimization of a prototype performs 100 iterations, each meta-iteration corresponds to 20 evaluations of the target benchmark. The cost of the feedback engine represents only 2% of the time spent on a 50 dimensional benchmark which amounts to 30 milliseconds on average (single core on *Intel(R) Core(TM) i5-8400 CPU @ 2.8 0GHz*). Most of this time is spent evaluating the objective functions. Each meta-optimization process takes, on average, 10 minutes to complete on a single core.

3.4 Comparison to reference algorithms

In this section, we assess the ability of the framework to produce efficient optimizers by comparison to other PSO variants such as constant PSO, Random Inertia Weight PSO (PSO-RIW) and Adaptive Velocity Informed PSO (APSO-VI) from Xu (2013). The code for our framework is available on Github (Roy (2023)). PSO-RIW and PSO were selected because they emerged as the best-performing options in the study conducted by Harrison et al. (2016). The selection of APSO-VI is grounded in similar considerations outlined in Harrison et al. (2018). Also included in this comparative benchmark are GA from Solgi (2018), Linear population size reduction Success-History Adaptive Differential Evolution (L-SHADE) from Tanabe and Fukunaga (2014), CMA-ES from Hansen et al. (2019). These algorithms provide a reference point to situate our adaptive PSO among well-known algorithms. The hyperparameters for those algorithms are available in Table 6. The population sizes were tuned on HAR[40] for L-SHADE[1] and GA as the defaults were sub-optimal. Parameters for PSO, APSO-VI and PSO-RIW were left to default (fully connected swarms) to keep consistency with Harrison et al. (2016, 2018). CMA-ES boundaries were enforced using bounds transformations (Biedrzycki (2020)) as it offered the best performance. The default population sizes were optimal. The commit hashes (Github version) are provided for reproducibility. All the codes used in these experiments are available on Github[2].

Table 7 demonstrates the capability of six FPSO prototypes highlighted earlier on

[1]https://github.com/Kaeryv/shady
[2]https://github.com/Kaeryv/FuzzySwarmBenchmarks

Strategy	Accuracy					Consistency				
	U	M	U+M	Rank	BRF	U	M	U+M	Rank	BRF
Constant PSO	-46	-144	-190	9	0	-40	-50	-90	9	1
L-SHADE	34	148	182	3	2	26	148	174	2	4
PSO-RIW	-66	-202	-268	10	0	-56	-104	-160	10	1
APSO-VI	-96	-192	-288	12	0	-100	-126	-226	11	0
CMA-ES	92	68	160	5	10	92	56	148	3	10
GA	-132	-148	-280	11	1	-132	-164	-296	12	0
FPSO$_1$: ω, c_1, u, K	**62**	**158**	**220**	**1**	**5**	**84**	**96**	**180**	**1**	**2**
FPSO$_2$: ω, l, u, K	76	110	186	2	0	62	56	118	5	0
FPSO$_3$: ω	4	44	48	7	0	14	44	58	7	0
FPSO$_4$: ω, c_1, l, K	16	90	106	6	0	26	36	62	6	0
FPSO$_5$: ω, c_1, h, K	64	112	176	4	0	54	66	120	4	1
FPSO$_6$: K	-8	-44	-52	8	1	-30	-58	-88	8	0

Table 7: **Accuracy and consistency achieved by each algorithm on (training) benchmark HAR[40].** Sum of scores are provided for unimodal (U) and multimodal (M) problems separately while the rank and BRF are computed on the global sum of scores (U+M). Best methods for each metric are highlighted in bold.

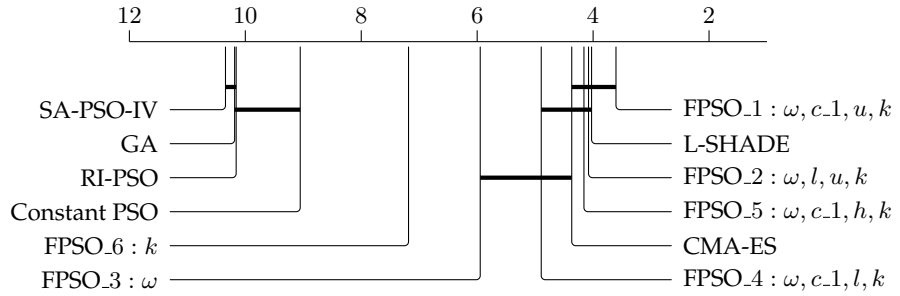


Figure 8: **Critical difference diagram for the accuracies (Acc) on benchmark HAR[40]** The methods are ranked from worst (12) to best (1). The black ties represent methods between which a U test failed to reject the null hypothesis, showing no significant distinction in performance.

benchmark HAR[40]. The prototypes were chosen as follows: FPSO$_{1,2}$ and FPSO$_{4,5}$ are the bests of benchmarks HAR[40] and CEC[50] respectively. While FPSO$_3$ and FPSO$_6$ are second and best best single-parameter prototypes of those same benchmarks. A first result of Table 7 is that all these FPSO variants and classic PSO performed better than PSO with random or adaptive inertia. FPSO variants that control 4 parameters have a clear lead on multimodal functions. CMA-ES remains the best performing method on unimodal problems. This statement is confirmed by Bonferroni-adjusted pairwise Wilcoxon-Mann-Whitney U tests (Wilcoxon (1945)) as illustrated by the critical difference diagram (CDD) in Figure 8 (Demšar (2006); Benavoli et al. (2016)): the black ties on Figure 8 show that FPSO$_{1,2,4,5}$ and even FPSO$_3$ have statistically indistinguishable accuracy to CMA-ES and L-SHADE. The lower accuracy of the two single-parameter controls corroborates Figure 7(A). The accuracy of single-parameter controllers for K and ω is already departing from the score of naive PSO and GA with a statistically significant difference from Figure 8. Results from Table 7 are reassuring about the ef-

ficiency of the meta-optimization and show that performance close to state-of-the-art algorithms can be achieved on training benchmark.

Table 8 and its associated CDD in Figure 9 further assess the validation performance over the functions of benchmark CEC^{50}. We notice the sample of controllers did not overspecialize the training benchmark. FPSO variants perform well again: the control enables PSO to compare with state-of-the-art alogrithms even with different functions and number of dimensions. FPSO$_{1,4,5}$ are even withing critical difference of L-SHADE.

The performance of CMA-ES cannot be beaten on this foreign benchmark and CMA-ES achieves the best accuracy rank on all functions. CMA-ES is way better than any other algorithm on unimodal problems like f_1 from CEC^{50} (Shifted and rotated bent cigar function) but also functions which present a steep valley on a large scale like Ackley, as shown in Figure 10(A). However, on complex multimodal problems (Figure 10(B,C)), the difference is a little less drastic.

This is a great step towards validation of the produced optimizers but we know nothing of their behavior: is it coherent with the understanding we have about parameters or does it just set parameters to a random or constant value? To answer these concerns, we will analyze the behavior of our algorithm in the two following sections.

Strategy	Accuracy			Consistency		
	U+M	Rank	BRF	U+M	Rank	BRF
Constant PSO	-108	9	0	-20	7	0
L-SHADE	140	2	3	**154**	**1**	**4**
PSO-RIW	-154	11	0	-20	8	0
APSO-VI	-152	10	0	-36	10	1
CMA-ES	**208**	**1**	**7**	94	2	3
GA	-196	12	0	-124	12	0
FPSO$_1$: ω, c_1, u, K	64	5	0	16	3	1
FPSO$_2$: ω, l, u, K	34	6	0	6	5	1
FPSO$_3$: ω	-4	8	0	-52	11	0
FPSO$_4$: ω, c_1, l, K	76	4	0	-2	6	0
FPSO$_5$: ω, c_1, h, K	78	3	0	16	4	0
FPSO$_6$: K	14	7	0	-32	9	0

Table 8: **Accuracy and consistency achieved by each algorithm on (validation) benchmark V (50D).**

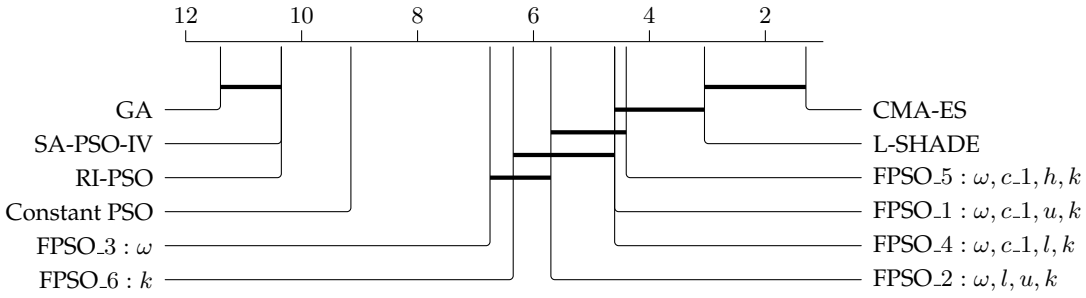


Figure 9: **Critical difference diagram for the accuracies (Acc) on benchmark CEC^{50}** The methods are ranked from worst (12) to best (1). The black ties represent methods between which a U test failed to reject the null hypothesis, showing no significant distinction in performance.

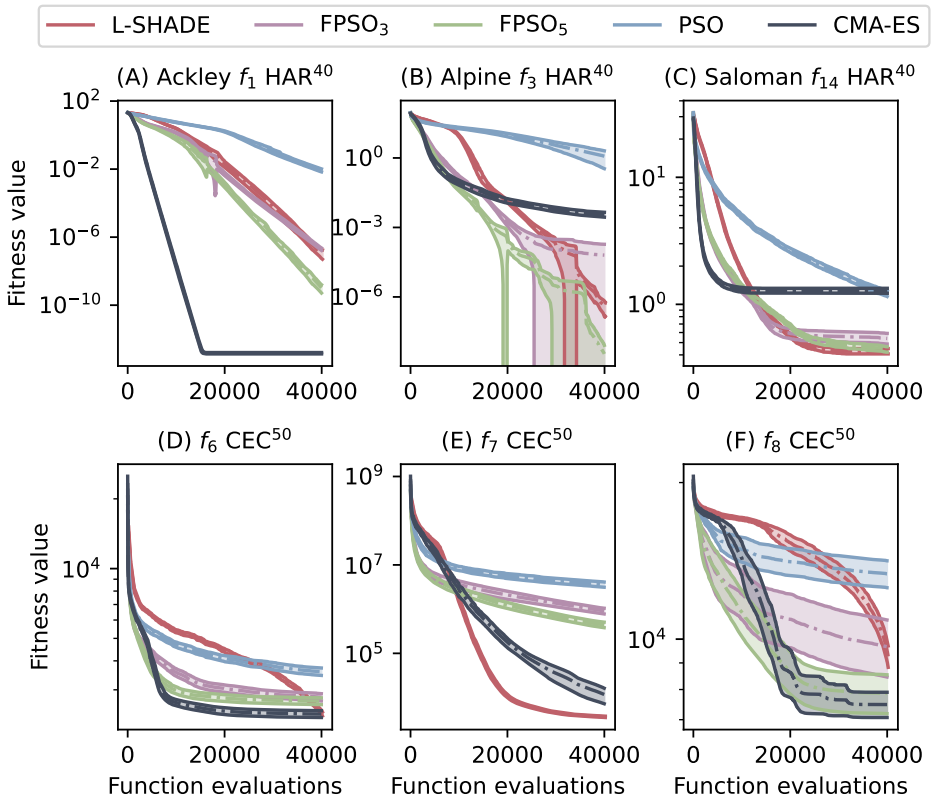


Figure 10: Convergence profiles for functions Ackley, Alpine and Saloman from HAR^{40}, and $f_{6,7,8}$ for CEC^{50}. The 95% confidence interval is shown in the background. The dotted curve is the mean.

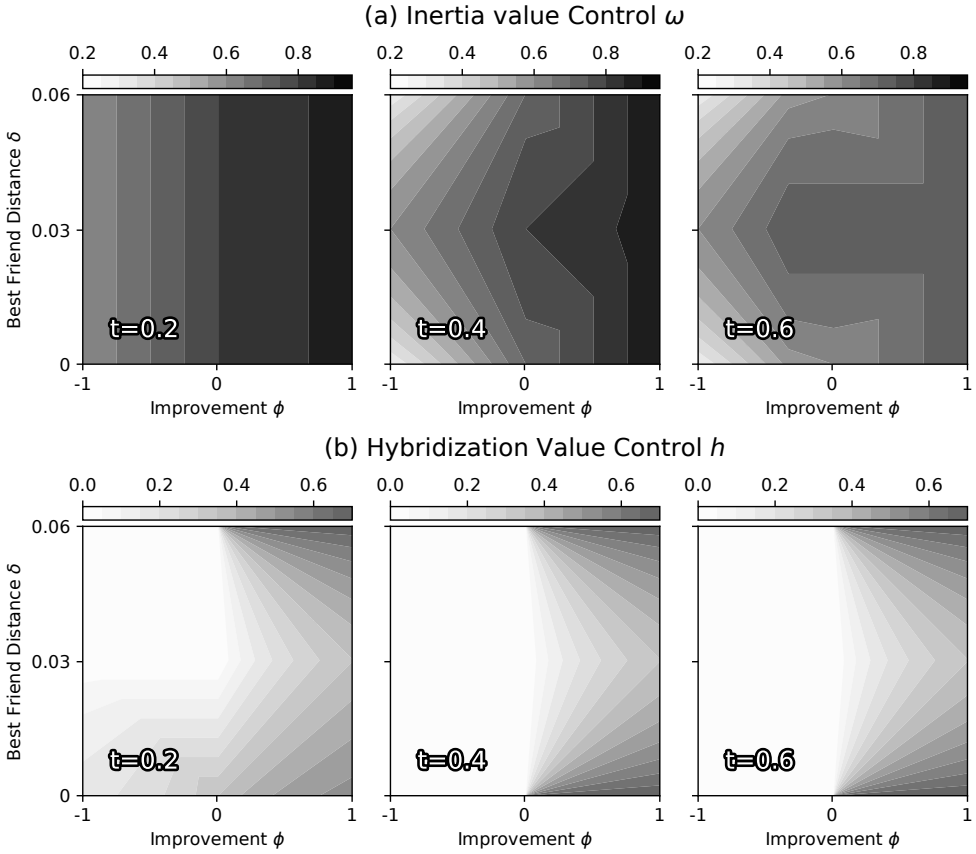


Figure 11: **Visualization of the control policies for h and ω in prototype (ω, h).** The value of the controlled parameter is represented on a map where the ordinates represent the distance to best friend (δ) and the abscissa the improvement (ϕ). Three snapshots were taken for three values of probe t (iteration).

4 Discussions

4.1 An example of control policy

The previous section showed that meta-optimization leads to well-performing optimizers. We now cover what is going on under the hood of a specifically trained instance of FPSO(ω, h). This controller was trained with fixed key values for the membership functions of ω and h to serve as illustration. In Figure 11 we show the dependence of ω and h on the three probes (ϕ, δ and t) for the controller of FPSO(h, ω). The adopted strategy boosts inertia in the early stages, particularly for particles improving their fitness value. In the later stages, inertia is lower. The controller for h uses quantum PSO for packed, stagnating agents in the beginning and for remote, improving agents. For the two later stages ($t \geq 0.4$), the controller only uses QPSO for improving particles. QPSO redistributes future agents' positions midway between the best memory and the best friend (see Sun et al. (2012), and Juan et al. (2004)). This behavior allows for a great motion range when either the best friend or memory is far away. In our implementa-

tion, the random adaptive topology often changes the best friend and leverages more exploration in this way. In later stages of optimization, when the best friend and memory tend to stay closer, improving particles are using QPSO to refine the best agents. QPSO is in fact fit when dealing with small local minima or discontinuities. QPSO narrows the search to a particular location by continuously sampling the neighborhood of the best solutions while PSO tends to converge to a fixed point. The different policies obtained for FPSO(h, ω) through meta-optimization follow mostly the same pattern, which seems consistent when discussed.

4.2 Fuzzy control and convergence observed through probes and parameters

Let us shift our focus to a prototype that regulates parameters widely examined in PSO literature: ω, c_1, and c_2. Its impact on the optimization process is assessed in comparison to a controller exclusively adjusting ω and, ultimately, a PSO with constant parameters. Our analysis will focus on the Ackley function with 50 dimensions, serving as an illustrative case.

Figure 12(A2,A3) displays the value of parameters ω, c_1 and c_2 in a way similar to Cleghorn and Engelbrecht (2014) for each iteration. This representation allows us to chart the convergence area derived by Poli (2009) in the background. Both of these parameter controls lead exclusively to convergent parameters on this single optimizer instance. However it is important to note that these observations are also reported for 97% of the iterations for all prototypes on all functions. These more thorough measurements used 10 independent executions for prototypes controlling at least ω, c_1 or c_2. Only one prototype leads to constantly divergent parameters while controlling ω and u. This violation was handled by a lower parameter u. Two prototypes lead to 50% of iterations with parameter values outside the prescribed area: the violations were less that 2% of the parameter values and appeared in the second half of the optimization process. This illustrative scenario is therefore representative of what these controllers can achieve.

Figure 12(B1,B2,B3) investigates the algorithm's exploitation and exploration dynamics through the lens of the probes. Without control, in Figure 12(B1), the optimizer stagnates to a local minimum (error $= 8.87$). The probe δ also indicates that the swarm fails to converge properly. When controlling inertia (ω) in Figure 12(B2), the swarm obtains a better optimal fitness and converges better as shown by $\delta_{\text{min}} \approx 10^{-8}$. We can see in Figure 12(C2) that this is due on one hand to a lower mean inertia and the controller choosing to rise this value at the beginning of the optimization process. The third case on the right of Figure 12 shows a slightly better control where all three parameters are controlled. We notice in Figure 12(C3) that the baseline cognitive factor is higher, providing exploration in the early stages and compensating for the lower base inertia again. This social factor rejoins the cognitive factor in the second half of the optimization, switching the swarm to more exploitation. The inertia is also reduced in the second half, allowing for even stronger exploitation.

We can conclude that the optimized algorithm further exploits the evaluation budget by exploring unlikely decision variables while remaining able to converge efficiently to exploit a promising optimum. FPSO, thanks to the feedback, is able to produce this kind of two-step dynamics where more room is left to exploration without sacrificing exploitation.

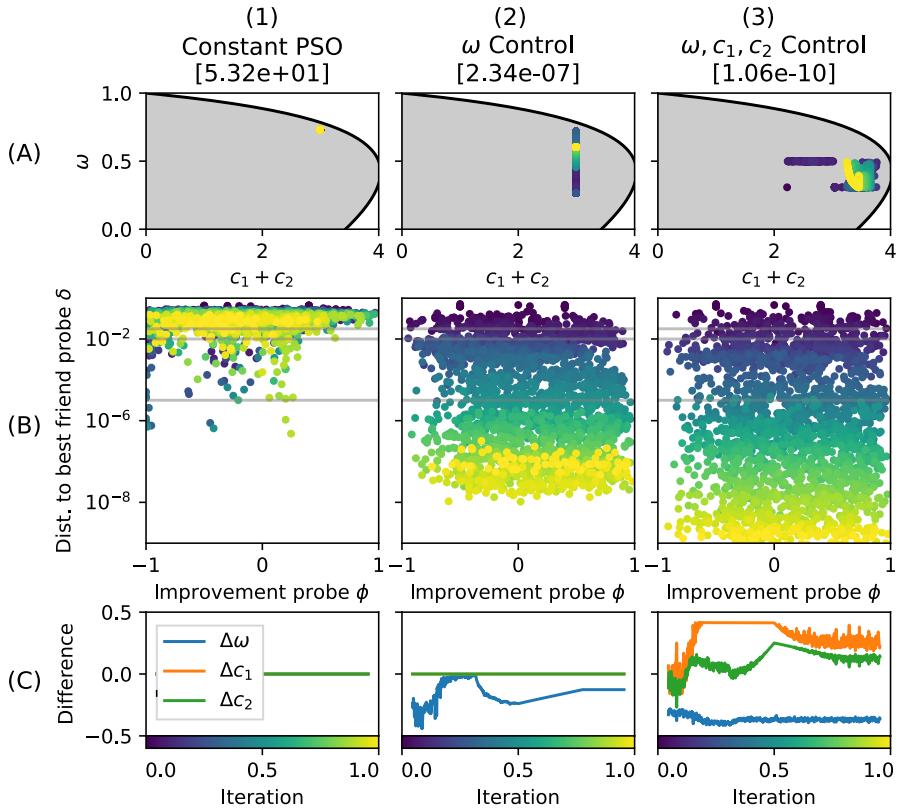


Figure 12: **Visualization of the swarm in probe and parameter spaces during an optimization.** The visualization is repeated from left to right for three control prototypes. The optimal fitness attained is shown below each title. (A) Parameter values plotted with the convergence area from Poli (2009) in the background. (B) The swarm in the space of probes for all iterations: the color represents the iterations, from start (blue) to finish (yellow). (C) Evolution of the parameter difference for ω, c_1 and c_2 to their respective canonical values (0.7298, 1.49618 and 1.49618).

4.3 Convergence assessment of FPSO

To discern divergent behavior among the optimized FPSOs, an empirical convergence criterion from Cleghorn and Engelbrecht (2014) was employed. The criterion investigates the average distance traveled by particles in a swarm optimizing a specific criterion function (CF). The function CF is a uniform noise defined as

$$\mathrm{CF}(\mathbf{x}, t) = \mathrm{U}(-1000, 1000),$$
$$\forall x, t_1, t_2, \mathrm{CF}(\mathbf{x}, t_1) = \mathrm{CF}(\mathbf{x}, t_2).$$

In this expression, $U(a, b)$ samples random numbers uniformly in the range $[a, b]$. This definition enforces that CF is deterministic: subsequent calls (at iterations t_1, t_2) for the same position must lead to the same value. The function has to be used without boundaries enforcement or speed limitations in the PSO algorithm. The initial speed

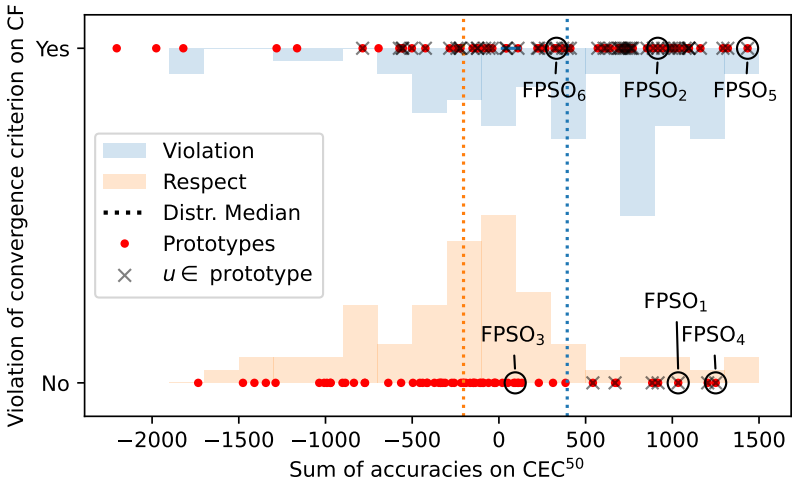


Figure 13: **Study of the modified convergence criterion for all FPSO prototypes.**

has to be set to zero. The average distance traveled per iteration is calculated as follows:

$$\bar{\Delta}(t+1) = \frac{1}{n_S} \sum_{i}^{n_S} \min \left[\, ||\mathbf{x}_i(t+1) - \mathbf{x}_i(t)| \, |_2, \; \Delta_{\max} \right].$$

Here, $n_S = 64$ represents the swarm's size, and $\mathbf{x}_i$ denotes the position of the ith particle. The optimization of CF is conducted over 5000 iterations. The values are capped by the limit $\Delta_{\max}$ like in Harrison et al. (2018) to avoid few highly divergent particles to take over the metric.

$$\Delta_{\max} = \sqrt{n_D \, (b_u - b_l)^2},$$

Here, $b_l = -100$ and $b_u = 100$ denote the lower and upper boundaries of the search space and $n_D = 100$ is the number of dimensions. The average distance traveled should remain lower than the threshold $\Delta_{\max}$ to qualify the swarm as convergent. This permissive criterion ($\Delta_{\max} = 2000$) is still able to discern divergent behavior as without speed limitation, divergent swarms often undergo velocity explosion: velocities several order of magnitudes larger than $\Delta_{\max}$ are reached. Considering the cap placed on $\Delta_{\max}$, we will consider that a divergent swarm is characterized by a $\bar{\Delta}$ closing on $\Delta_{\max}$ at the end of the optimization process on CF.

The criterion requires not limiting speed and not imposing boundaries. However speed limitations are an integral part of many control policies through parameters l and u. The speed limitations enforce convergence to some extent that the original criterion wants removed. Here, we chose to let down the upper speed limit to respect the conditions of the criterion. This decision will however greatly affect prototypes relying on u.

The results from the convergence study are summarized in Figure 13. Prototypes were considered convergent when more than 50% of their last 100 iterations had a Δ under $0.85\Delta_{\max}$. The fraction 0.85 was tuned to split the prototypes in two groups of the same size: those violating the criterion and those respecting the criterion. This criterion

aligns with the rationale we just discussed: a swarm that undergoes velocity explosion will have a Δ approaching Δ_{max}. All 127 prototypes are represented in Figure 13 by red dots. The ones respecting the criterion are found below while the ones violating the criterion are found above. The abscissa represent the sum of accuracies Σ_{Acc} on benchmark CEC^{50}. Their normalized distributions are represented in orange and blue respectively.

From their distributions, we notice that the criterion $0.85\Delta_{\text{max}}$ splits the prototypes in two distinct groups in terms of performance on benchmark CEC^{50}. This assertion is confirmed through a Mann-Whitney U test with a significance level of $4\text{e}{-5}$. Surprisingly the group violating the criterion has a significantly better median rank that the other. Two leads may explain this contradictory result. Firstly, the disabling of parameter u, shown to be well-performing will certainly lead a prototype trained in its presence to misbehave on CF. In fact, we confirm this result empirically as 89% of prototypes including u are classified as diverging under this criterion (black crosses in Figure 13) while 90% of the other prototypes are convergent. Secondly, the dynamic nature of parameter control should also be considered: a prototype may first converge, then reactivate exploration to reach neighboring minima. This kind of behavior would be fatal for convergence on CF while not harmful for the optimizer's performance. Finally it is noteworthy that the convergence criterion still identified that the worst prototypes on the benchmark were also divergent.

Figure 13 also indicate how our selection of prototypes performed on this criterion. Half of them respected the criterion: FPSO$_{1,4,6}$, indicating robustness even to the loss of a controlled parameter in the case of FPSO$_4$.

4.4 Adapting across different fitness landscapes and number of dimensions

In Figure 14, we present the pairwise Spearman rank correlation coefficients among all benchmark listed in Table 4. The Spearman correlation coefficient, denoted as ρ, gauges the rank correlation between the accuracies of prototypes on two benchmarks, effectively identifying any monotonic relationship between them. The Spearman test requires rejecting the null hypothesis: we used a significance level of 0.05. Benchmark pairs that failed to match this criterion were hatched. Thus, Figure 14 serves as an indicator of the transfer of optimization ability between benchmarks. Particularly, the lines containing HAR^{40} (the training benchmark) assess the performance transfer of the training procedure. A positive correlation between two benchmarks implies that superior prototypes on one benchmark are likely to perform well on the other, and vice versa. Analyzing the Spearman correlation map reveals a strong correlation among the performances of Benchmarks HAR^{10,20,40} and a slightly but positively correlated relationship with Benchmark CEC^{50}. This observation underscores the feasibility of transferring optimization knowledge acquired from one benchmark to another. Nonetheless, a constraint exists wherein either the dimension or the fitness landscape must remain consistent, or else the correlation breaks down. Indeed, validation benchmark CEC^{10,20} exhibits a mild anticorrelation with the training benchmark, suggesting that the training reaches its capacity for these dimensions. We observed that the prototypes, trained on 40 dimensions, tend to favor high exploration: this makes the method less efficient on more simpler problems where speed of convergence is key. This limitation could be challenged by using benchmarks of different dimensions while training.

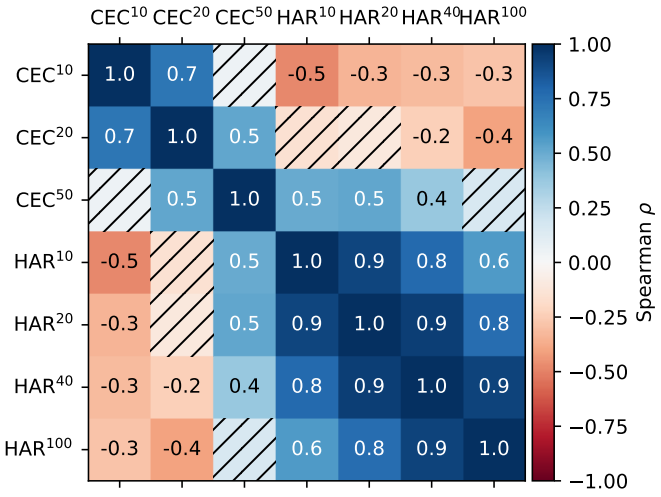


Figure 14: **Pairwise Spearman correlation coefficient between Benchmarks HAR^{10,20,40,100} and CEC^{10,20,50}.** A positive correlation ρ is shown in blue while negative correlations are shown in red. Whenever a test failed to reach a significance level of 0.05, the corresponding cell was hatched.

5 Performance assessment on real-world test cases

In this last section, the best FPSOs highlighted in previous sections are compared to CMA-ES on the real world Benchmark RW from CEC$_{2011}$ (Das and Suganthan (2011)). RW$_1$ is a six dimensional problem where a Frequency-Modulated sound wave is synthesized. The objective is to generate a sound similar to a target sound. The problem is multimodal with a global minima $f(x_{\text{best}}) = 0$. RW$_2$ minimizes a Lennard-Jones potential to conform a icosahedral molecular structure. This problem has 30 dimensions and comprises a sharp fitness landscape as well as numerous local minima. RW$_{3,4}$ are two one-dimensional, highly multimodal optimal control problems: a bifunctional catalyst blend and a non-linear stirred tank reactor. In the first case, the decision variable is the mass fraction of an hydrogenation catalyst for converting methylcyclopentane to benzene. In the second one, we decide on the flow rate of a cooling fluid during the process. RW$_{5,6}$ are two minimizations of Tersoff potential function for Silicon ($n_D = 30$). RW$_7$ is a radar system design problem that involves selecting an appropriate waveform for pulse compression ($n_D = 20$). The specific method proposed by Dukic and Dobrosavljevic is characterized as a challenging min–max nonlinear non-convex continuous optimization problem. Finally, the last case RW$_{10}$ is a circular antenna design problem ($n_D = 12$). The six antenna elements are placed on a circle and their phase is optimized in order to suppress side-lobes and minimize beam width.

Results on these benchmark are shown in Table 9. We consider two converging FPSO variants here: the complex FPSO$_4(\omega, c_1, l, K)$ and the simple FPSO$_3(\omega)$. For this benchmark, we will show the fitness values as they are more meaningful for real-world problems. In particular, we are interested in the best value one can get by using 30 independent runs. The values obtained in our experiments are compared to the opti-

	PSO		$FPSO_3$		$FPSO_4$		CMA-ES		GA-MPC	
RW_1	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	5.0e-08	1.4e+01	**0.0e+00**	**1.2e+01**	4.5e-26	1.4e+01	1.2e+01	2.4e+01	0.0e+00	0.0e+00
1e+05	6.4e-30	2.2e+00	0.0e+00	1.2e+01	1.3e-26	1.1e+01	9.6e-16	2.0e+01	0.0e+00	0.0e+00
RW_2	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	-1.8e+01	-1.4e+01	-2.7e+01	-2.5e+01	**-2.8e+01**	**-2.6e+01**	-2.7e+01	-2.0e+01	-2.8e+01	-2.8e+01
1e+05	-2.3e+01	-1.8e+01	-2.8e+01	-2.6e+01	-2.8e+01	-2.7e+01	**-2.7e+01**	**-2.7e+01**	-2.8e+01	-2.8e+01
RW_3	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	1.2e-05	1.2e-05	1.2e-05	1.2e-05	**1.2e-05**	**1.2e-05**	n/a	n/a	1.2e-05	1.2e-05
1e+05	1.2e-05	1.2e-05	1.2e-05	1.2e-05	**1.2e-05**	**1.2e-05**	n/a	n/a	1.2e-05	1.2e-05
RW_4	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	1.4e+01	1.4e+01	1.4e+01	1.4e+01	1.4e+01	2.1e+01	n/a	n/a	1.4e+01	1.4e+01
1e+05	1.4e+01	1.4e+01	1.4e+01	1.4e+01	1.4e+01	2.1e+01	n/a	n/a	1.4e+01	1.4e+01
RW_5	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	-3.4e+01	-2.8e+01	**-3.5e+01**	**-3.3e+01**	-3.5e+01	-3.2e+01	-3.7e+01	-3.3e+01	-3.7e+01	-3.5e+01
1e+05	-3.7e+01	-3.0e+01	**-3.7e+01**	**-3.3e+01**	-3.6e+01	-3.2e+01	-3.4e+01	-3.1e+01	-3.7e+01	-3.5e+01
RW_6	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	-2.3e+01	-2.0e+01	**-2.9e+01**	**-2.3e+01**	-2.7e+01	-2.0e+01	-1.9e+01	-1.3e+01	-2.9e+01	-2.8e+01
1e+05	-2.3e+01	-2.1e+01	**-2.7e+01**	**-2.3e+01**	-2.7e+01	-2.0e+01	-2.0e+01	-1.6e+01	-2.9e+01	-2.8e+01
RW_7	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	1.3e+00	1.5e+00	8.9e-01	1.3e+00	9.1e-01	1.3e+00	**5.0e-01**	**6.8e-01**	5.0e-01	7.5e-01
1e+05	1.2e+00	1.4e+00	8.3e-01	1.2e+00	8.4e-01	1.1e+00	**5.0e-01**	**6.4e-01**	5.0e-01	7.5e-01
RW_{10}	Min	Med.	Min	Med.	Min	Med.	Min	Med.	Min	Med.
5e+04	-2.2e+01	-1.9e+01	**-2.1e+01**	**-2.1e+01**	-2.2e+01	-2.1e+01	-2.2e+01	-1.6e+01	-2.2e+01	-2.2e+01
1e+05	-2.2e+01	-1.9e+01	**-2.1e+01**	**-2.1e+01**	-2.2e+01	-2.1e+01	-2.2e+01	-1.9e+01	-2.2e+01	-2.2e+01

Table 9: **Minimal and median fitness values for $5e4$ and $1e5$ evaluations of real-world problems from CEC 2011.** Methods in bold achieved the best median performance on the problem (GA-MPC is not included). The cells filled with n/a are due to the CMA-ES package not supporting 1D problems.

mal values obtained by the winner of the competition : Genetic Algorithm with a new multi-parent crossover (GA-MPC, Elsayed et al. (2011)).

From Table 9 we observe that $RW_{1,2,6,7}$ differentiate the most the different methods. For RW_1, every method except CMA-ES reaches at least on one run the optimal fitness of 0.0 with as few as $5e4$ evaluations. The median fitness is also higher for CMA-ES than for PSO. On RW_2, all methods except constant PSO have performance comparable to GA-MPC. On problem RW_6, PSO again outperforms CMA-ES. However, on the radar system (RW_7), CMA-ES is the clear leader, having similar performance to GA-MPC. Finally, for the circular antenna design (RW_{10}), all methods achieved similar performance but FPSO variants achieved better median performance. We conclude from Table 9 that $FPSO_3$ stands out as a particularly reliable and well performing PSO variant.

6 Conclusions

The meta-optimization framework discussed in this work proved to be able to produce adaptive FPSO reliable on our training benchmark as well as on the CEC 2020 benchmark. These observations were also extended to benchmarks sharing the same functions or numbers of dimensions. The produced FPSO performed well on the target benchmark, going often beyond the performance of the much more complex optimizer CMA-ES. The adaptive optimizer achieves its objectives without succumbing to overfitting on the training functions. Remarkably, substantial and consistent improvements were also noted in comparison to PSO when applied to the validation benchmarks. Although the gains were more modest on these foreign fitness landscapes, the meta-

optimized controllers beat by a large margin PSO-RIW and APSO-IV controllers. These algorithms were proved to include the best parameter controllers.

We finally showed that the behavior introduced in the swarm by the feedback made perfect sense. The change in behaviour was observed in the different probes designed as input for the feedback of PSO. Convergence of the produced FPSOs was also studied with an adapted criterion. Although our results were inconclusive due to technical incompatibilities between some controllers and the conditions of the criterion, we provided a sound statistical overview of the criterion on all FPSOs. The overview showed that controllers fairly studied respected the criterion. However, it also showed that the respect of the criterion wasn't linked to better performance on the validation benchmark.

The feedback has a tiny impact on the algorithm, both in terms of numerical speed and implementation. The fuzzy rule sets are compact, single-integer programs, quickly evaluated, that interface with the algorithm through its parameters and some probes.

Future work could apply this framework to state-of-the-art algorithms and explore systematically the potential benefits of adaptive parameters tuning. We seek to continue this research with the incorporation of the proposed meta-optimization as an advanced restart strategy for the controlled algorithm. When a consequent number of evaluations is available, the meta-optimization can be applied directly to the targeted problem instead of a training benchmark. The method would then become self-adaptive with online tuning of the controllers themselves. We finally hope that this work will motivate systematic approaches to the design of adaptive parameters policies. It is a great way to gain insight on the algorithm parameters while improving its performance. Theoretical criteria can be studied in this way because a large number of different policies can be generated either with a set goal or randomly. The framework is as such a great tool for empirical investigation of theoretical developments.

7 Acknowledgments

Computational resources have been provided by the Consortium des Équipements de Calcul Intensif (CÉCI), funded by the Fonds de la Recherche Scientifique de Belgique (F.R.S.- FNRS) under Grant No. 2.5020.11 and by the Walloon Region. The authors also thank Pr. Timotéo Carletti and Dr. Delphine Nicolay from the University of Namur and Anthony Hendricks, former research engineer at Cenaero for their valuable insights in the early stages of the project.

References

Adorio, E. P. and January, R. (2005). MVF - Multivariate test functions library in C for unconstrained global optimization.

Al-Roomi, A. R. (2015). Unconstrained Single-Objective Benchmark Functions Repository.

Bansal, J. C., Singh, P. K., Saraswat, M., Verma, A., Jadon, S. S., and Abraham, A. (2011). Inertia weight strategies in particle swarm optimization. In *2011 Third World Congress on Nature and Biologically Inspired Computing*, pages 633–640.

Bartz-Beielstein, T., Parsopoulos, K., and Vrahatis, M. (2002). Tuning PSO parameters through sensitivity analysis.

Bedau, M. A. (1997). Weak emergence. *Philosophical perspectives*, 11:375–399.

Benavoli, A., Corani, G., and Mangili, F. (2016). Should we really use post-hoc tests based on mean-ranks? *The Journal of Machine Learning Research*, 17(1):152–161.

Biedrzycki, R. (2020). Handling bound constraints in cma-es: An experimental study. *Swarm and Evolutionary Computation*, 52:100627.

Burke, E., Kendall, G., Newall, J., Hart, E., Ross, P., and Schulenburg, S. (2003). *Hyper-Heuristics: An Emerging Direction in Modern Search Technology*.

Cleghorn, C. W. and Engelbrecht, A. P. (2014). Particle swarm convergence: An empirical investigation. In *2014 IEEE Congress on Evolutionary Computation (CEC)*, pages 2524–2530. IEEE.

Cleghorn, C. W. and Engelbrecht, A. P. (2015). Particle swarm variants: standardized convergence analysis. *Swarm Intelligence*, 9:177–203.

Cowling, P., Kendall, G., and Soubeiga, E. (2002). Hyperheuristics: A tool for rapid prototyping in scheduling and optimisation. In *Applications of Evolutionary Computing: EvoWorkshops 2002: EvoCOP, EvoIASP, EvoSTIM/EvoPLAN Kinsale, Ireland, April 3–4, 2002 Proceedings*, pages 1–10. Springer.

Das, S. and Suganthan, P. N. (2011). Problem definitions and evaluation criteria for cec 2011 competition on testing evolutionary algorithms on real world optimization problems.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *The Journal of Machine learning research*, 7(1):1–30.

Doerr, B. and Doerr, C. (2020). *Theory of Parameter Control for Discrete Black-Box Optimization: Provable Performance Gains Through Dynamic Parameter Choices*, pages 271–321. Springer International Publishing, Cham.

Eiben, A., Hinterding, R., and Michalewicz, Z. (1999). Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3(2):124–141.

Elsayed, S. M., Sarker, R. A., and Essam, D. L. (2011). Ga with a new multi-parent crossover for solving IEEE-CEC2011 competition problems. In *2011 IEEE Congress of Evolutionary Computation (CEC)*, pages 1034–1040.

Erdős, P. and Rényi, A. (1959). On random graphs. I. *Publ. Math. Debrecen*, 6:290–297.

Hansen, N., Akimoto, Y., and Baudis, P. (2019). CMA-ES/pycma on Github. Zenodo, DOI:10.5281/zenodo.2559634.

Hansen, N. and Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195.

Harrison, K. R., Engelbrecht, A. P., and Ombuki-Berman, B. M. (2016). Inertia weight control strategies for particle swarm optimization: Too much momentum, not enough analysis. *Swarm Intelligence*, 10:267–305.

Harrison, K. R., Engelbrecht, A. P., and Ombuki-Berman, B. M. (2018). Self-adaptive particle swarm optimization: a review and analysis of convergence. *Swarm Intelligence*, 12:187–226.

Hawkins, D. M. (2004). The problem of overfitting. *Journal of Chemical Information and Computer Sciences*, 44(1):1–12. PMID: 14741005.

Holland, J. (1992). *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. Complex Adaptive Systems. MIT Press, Cambridge, MA, USA.

Izquierdo, S. S. and Izquierdo, L. R. (2018). Mamdani fuzzy systems for modelling and simulation: A critical assessment. *Journal of Artificial Societies and Social Simulation*, 21(3):2.

Jamil, M. and Yang, X. S. (2013). A literature survey of benchmark functions for global optimisation problems. *International Journal of Mathematical Modelling and Numerical Optimisation*, 4(2):150.

Juan, L., Feng, B., and Xu, W. (2004). Particle swarm optimization with particles having quantum behavior. volume 1, pages 325–331.

Kennedy, J. and Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95-International Conference on Neural Networks*, volume 4, pages 1942–1948. IEEE.

Leonard, B. J. and Engelbrecht, A. P. (2013). On the optimality of particle swarm parameters in dynamic environments. In *2013 IEEE Congress on Evolutionary Computation*, pages 1564–1569.

Močkus, J. (1975). On bayesian methods for seeking the extremum. In *Optimization Techniques IFIP Technical Conference*, pages 400–404. Springer.

Nickabadi, A., Ebadzadeh, M. M., and Safabakhsh, R. (2011). A novel particle swarm optimization algorithm with adaptive inertia weight. *Applied Soft Computing*, 11(4):3658–3670.

Poli, R. (2009). Mean and variance of the sampling distribution of particle swarm optimizers during stagnation. *IEEE Transactions on Evolutionary Computation*, 13(4):712–721.

Rosenbrock, H. H. (1960). An Automatic Method for Finding the Greatest or Least Value of a Function. *The Computer Journal*, 3(3):175–184.

Roy, N. (2023). Hybris: Evolvable Fuzzy Particle Swarm Optimization. `https://github.com/Kaeryv/Hybris`.

Shi, Y. and Eberhart, R. (1998). A modified particle swarm optimizer. In *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No. 98TH8360)*, pages 69–73. IEEE.

Shi, Y. and Eberhart, R. (2001). Fuzzy adaptive particle swarm optimization. In *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546)*, volume 1, pages 101–106.

Solgi, R. (2018). Geneticalgorithm. `https://github.com/rmsolgi/geneticalgorithm`.

Storn, R. and Price, K. (1995). Differential evolution: A simple and efficient adaptive scheme for global optimization over continuous spaces. *Journal of Global Optimization*, 23.

Sugeno, M. (1974). Theory of fuzzy integrals and its applications. *Doct. Thesis, Tokyo Institute of technology*.

Sun, J., Fang, W., Wu, X., Palade, V., and Xu, W. (2012). Quantum-Behaved Particle Swarm Optimization: Analysis of Individual Particle Behavior and Parameter Selection. *Evolutionary Computation*, 20(3):349–393.

Tanabe, R. and Fukunaga, A. S. (2014). Improving the search performance of shade using linear population size reduction. In *2014 IEEE Congress on Evolutionary Computation (CEC)*, pages 1658–1665.

Tian, D.-p. and Li, N.-q. (2009). Fuzzy particle swarm optimization algorithm. In *2009 International Joint Conference on Artificial Intelligence*, pages 263–267.

Trelea, I. C. (2003). The particle swarm optimization algorithm: convergence analysis and parameter selection. *Information Processing Letters*, 85(6):317–325.

van den Bergh, F. and Engelbrecht, A. (2006). A study of particle swarm optimization particle trajectories. *Information Sciences*, 176(8):937–971.

Wang, F., Zhang, H., and Zhou, A. (2021). A particle swarm optimization algorithm for mixed-variable optimization problems. *Swarm and Evolutionary Computation*, 60:100808.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.

Wolpert, D. H. and Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82.

Xin, J., Chen, G., and Hai, Y. (2009). A particle swarm optimizer with multi-stage linearly-decreasing inertia weight. In *2009 International Joint Conference on Computational Sciences and Optimization*, volume 1, pages 505–508. IEEE.

Xu, G. (2013). An adaptive parameter tuning of particle swarm optimization algorithm. *Applied Mathematics and Computation*, 219(9):4560–4569.

Yue, C., Price, K., Suganthan, P., Liang, J., Ali, M., Qu, B., Awad, N., and Biswas, P. P. (2020). Problem definitions and evaluation criteria for the CEC 2020 special session and competition on single objective bound constrained numerical optimization.

Zadeh, L. (1965). Fuzzy sets. *Information and Control*, 8(3):338–353.

Zadeh, L. A. (1988). Fuzzy logic. *Computer*, 21(4):83–93.

Zambrano-Bigiarini, M., Clerc, M., and Rojas, R. (2013). Standard particle swarm optimisation 2011 at cec-2013: A baseline for future pso improvements. In *2013 IEEE Congress on Evolutionary Computation*, pages 2337–2344.